GECKO: an explainable system for designing keyboard layouts and the keyboards they run on

One scorer, one hand model, and one reproducible pipeline from a text corpus and a hand to a ranked layout and a printable board.

Joshua Markle

University of Virginia · October 2026

Draft · figures marked Template and numbers marked TODO are placeholders for runs not yet made

Abstract

Keyboard design is two coupled problems. Placing 30 letters is a combinatorial search ($30! \approx 2.65 \times 10^{32}$ layouts) wrapped around a judgement problem: what makes typing comfortable? Shaping the board those letters sit on is a continuous search over key positions and angles that changes the answer to the first problem. Existing tools judge layouts with one fixed, opaque score on one flat board, and board generators do not judge typing at all. GECKO is an open-source system (a Rust core, a command-line tool, an HTTP server, a WebAssembly build, and a TypeScript web UI with a 3D viewer) that treats both problems with one objective. A *scorer* folds a board geometry, a hand model, a text corpus, and a weighted profile of 48 metrics into cost tables, so a key swap costs 1.6 µs on one core and a search runs 5.0×10^6 swaps per second on a laptop. Nine layout optimizers share one trait and one runner; lower bounds, exact solutions of sub-problems, local-optima atlases, quality-diversity maps, and Pareto fronts say how good a result is and what else is nearly as good. A hand model with finger and thumb inverse kinematics turns board shape into cost, so a CMA-ES outer loop can search 2D and 3D boards, and free-form key poses, with the layout inside; a collision and manufacturability check keeps every board buildable, and the best board is written as a printable plate and case. Every run is seeded, hashed, and re-verifiable. We describe the design and the invariants that keep it correct, measure its speed and search quality, and walk through eight uses, from auditing a layout to printing a board fitted to one person's hand.

1. Introduction

Alternative layouts (Dvorak, Colemak, Workman, Graphite, Sturdy, Semimak, Canary, and thousands more in community collections) are designed with a family of analyzers and optimizers: carpalx, KLO, Kühlmak, genkey, Oxeylyzer, and cmini's analyzer, among others. Ergonomic boards (the Corne, Kyria, Dactyl-Manuform, and their many forks) are designed with a different family: ergogen, Cosmos, and hand-edited CAD. The two families never meet. Four problems recur.

The score is opaque. One number hides which trade-offs it made, and two tools rank the same layouts differently with no way to tell why. **The optimizer is welded to the objective.** Changing a metric means editing the search. **Nobody says how good ``good'' is.** A heuristic returns a layout, with no certificate and no map of what else was nearly as good. **The board is fixed.** Layout tools score a flat staggered grid; board tools shape keys by eye. Neither asks which board, for which hand, makes the best layout possible.

GECKO is built for four kinds of user: people choosing or designing a layout, and a board, for their own hands, language, and work; researchers who need a judge they can inspect and reproduce; keyboard builders who want to compare and generate boards; and scripts and agents, for which every command has a stable JSON output. Its contributions are:

- a scorer built for swaps: all metrics fold into one table per n-gram class, so adding a metric to a table costs nothing at search time (Section 4);
- one trait for the discrete optimizers, compared on equal terms by a benchmark harness with performance profiles and time-to-target curves (Section 5);
- explanations, certificates, and maps: every score decomposes per metric, finger, n-gram, and word; bounds and exact sub-problem solutions give optimality gaps; atlases, local-optima networks, and quality-diversity grids show what else is nearly as good (Section 6);
- geometry in the loop: a hand model whose costs

fold into the same tables, and a bilevel search over 2D, 3D, and free-form boards, finger assignments, and thumb clusters (Section 7);
- from model to print: collision and manufacturability checks and a generated switch plate and case (Section 8);
- eight worked uses (Section 9) and reproducibility by construction (Section 11).

2. System overview

GECKO is a Cargo workspace of three crates and a web app. `gecko-core` (about 24k lines of Rust) holds everything that matters: geometry, hand model, corpus, metrics, scorer, optimizers, bounds, geometry search, and mesh generation. It does no I/O beyond reading and writing data files. `gecko-cli` is the `gecko` binary, including `gecko serve`, an HTTP server with a JSON API and server-sent progress events. `gecko-wasm` wraps the same core for the browser, so the web UI scores a layout, poses the hand, and builds the case mesh with the code the CLI uses. The web UI (TypeScript, React, Three.js) has an input side (Board, Hand, Language, Profile, Search, Summary) and a results side (Home, Explore, Analysis, Saved, Export), with a live layout editor and a 3D view of the board and the posed hand. Its Home screen is the workflow in one picture (Figure 1): every input box opens its tab and shows what the next run will use, and every optimize mode reads the same scoring model and writes the same kind of run.

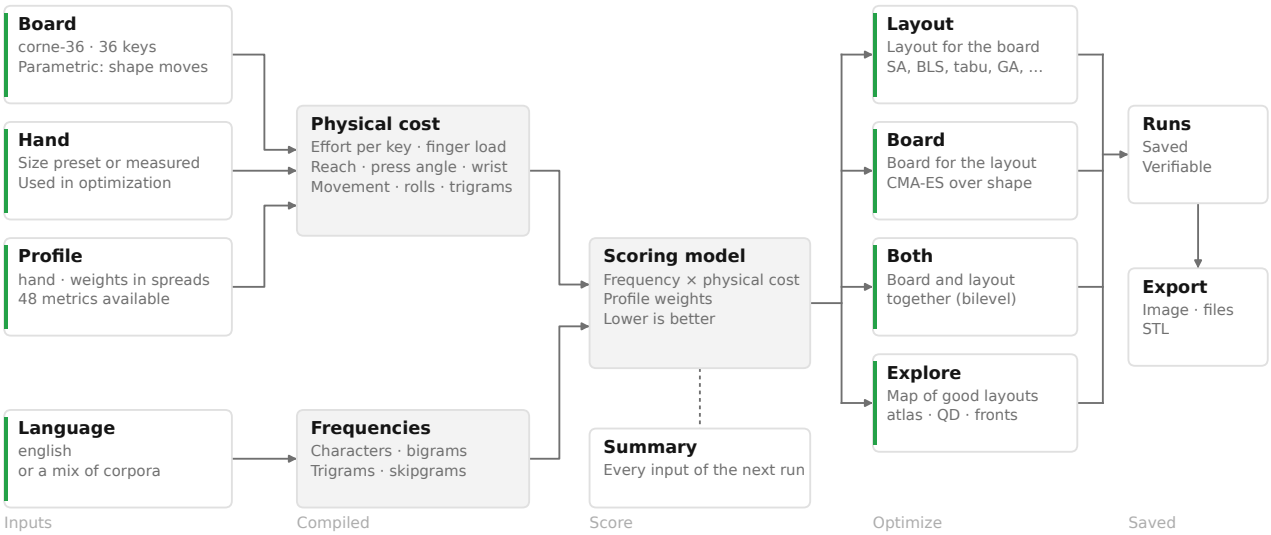


Figure 1: The web UI's Home screen. Inputs (left) compile into a physical cost per key and symbol frequencies; the scoring model combines them under the profile's weights; four optimize modes search it (a layout for a fixed board, a board for a fixed layout, both together, or a map of good layouts); every mode saves a verifiable run that can be exported as an image, layout files, or STL. Green bars mark inputs ready to run; tinted boxes are derived and have no tab. Drawn from `web/src/components/views/FlowChart.tsx` with the bundled defaults.

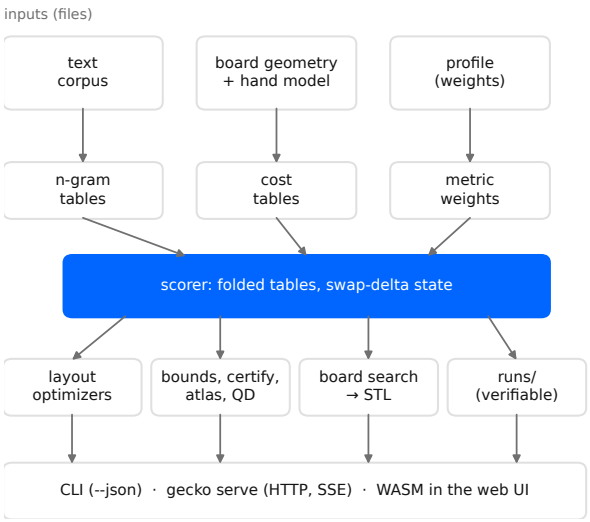


Figure 2: Data flow. A corpus becomes n-gram tables; a geometry, posed by the hand model, becomes cost tables; the profile weights them. The scorer folds these once, and every algorithm and front end reads it.

Inputs are five kinds of file. A *geometry* (TOML; explicit, or parametric in 2D and 3D; importable from keyboard-layout-editor.com) lists key slots with pose, finger, and effort. A *hand* gives segment lengths, joint limits, and arm geometry, from a size preset or a measured hand. A *corpus* is text compiled to a binary n-gram table (`.gkc`); bundled corpora include English, French, code, and Reddit text, and corpora can be mixed with weights. A *profile* (TOML) gives metric weights in spread units (Section 4), finger-load targets, and build limits; seven are bundled (default, code, sfb, engram, familiar, hand, learned). A *layout* is a slot-ordered list of symbols, optionally with layers and adaptive keys. Any command accepts a layout as a name, a TOML path, a literal (`raw:qwerty…`), a saved run (`run:<id>#rank`), or an imported community layout (`cmini:<name>`).

Outputs are run directories. Every search writes `runs/<time>-<algo>-<hash>/` with its configuration, input hashes, progress and telemetry logs, and ranked results; board searches add the best board as a loadable geometry and, on request, its STL files (Section 11). The web UI lists, compares, replays,

and re-opens them, so a result found on the command line can be inspected in 3D and vice versa.

3. Problem formulation

A *geometry* $G(\theta)$ is a list of N slots (pose, size, finger, row, column, home flag, effort), generated from parameters θ. A *layout* p assigns a symbol to each slot; empty slots, thumb tokens, and layer keys are allowed. A *corpus* supplies frequencies f_1 (symbols), f_2 (bigrams), f_2^k (skipgrams, gaps 1–3), and f_3 (trigrams). A *profile* holds metric weights and parameters. With $p(x)$ the slot of symbol x, the objective is

$$\begin{aligned}\text{score}(p,\theta) = {} & \textstyle\sum_x f_1[x]\, C_1^\theta[p(x)] \\ & + \textstyle\sum_{x,y} f_2[x,y]\, C_2^\theta[p(x),p(y)] + \text{skipgrams} \\ & + \textstyle\sum_{x,y,z} f_3[x,y,z]\, C_3^\theta[p(x),p(y),p(z)] \\ & + \text{non-linear terms} + \lambda\,\text{collide}(\theta).\end{aligned}$$

For a fixed board the first three lines are a quadratic assignment problem (QAP) with a sparse cubic part. The non-linear terms are functions of a few per-finger accumulators (finger balance, peak load, hand balance). Swapping symbols a and b changes only terms that contain a or b. The board enters only through the tables C^θ and a collision penalty, so the layout search never sees geometry, and a board search is an outer loop that rebuilds tables.

4. The scorer

4.1 Metrics as cost tables

A metric is a *table builder*: given a geometry and hand it returns a per-slot, per-pair, or per-triple cost, plus a rule for reporting a raw value (a percentage or a per-1000 rate). GECKO implements 48 metrics in families: same-finger (SFB, SFS, far SFS, distance); stretch and scissors (a geometric stretch adapted from Oxeylyzer 2, a finger-pair movement table); rolls and alternation (a dozen trigram classes, including thumbs); load and balance; comfort judges (the Engram formula and measured tables); hand-model costs (reach, wrist posture, press angle, thumb fit); and symbol and layer metrics (layer switches, code bigrams, paired brackets, home-row-mod misfires, shortcut placement, familiarity to a reference layout). The full list, with definitions and sources, is `gecko list metrics`.

4.2 Folding

For a profile with weights w_m, the linear metrics are summed once at build time, for example $C_2[s][t] = \sum_m w_m\, c_{2,m}[s][t]$, and likewise for C_1, the skipgram table, and C_3. Search cost therefore does not depend on how many metrics feed a table. Weights are expressed in *spread units*: each is divided by the metric's standard deviation over the reference layouts, so a weight reads as ``how many reference spreads of this metric I trade for one of another.'' Raw, unweighted values are recomputed separately for reports. Profiles can be written by hand, normalized, or fitted to a judge metric or to pairwise preferences (`gecko profile fit`).

4.3 Swap-delta evaluation

For a swap of a and b, the change is computed from the $O(S)$ bigram partners of each symbol and the sparse list of non-zero trigrams that contain them. Trigrams are stored per symbol; a `State` caches the folded cost of every non-zero trigram, so a delta reads the old cost and `apply` refreshes only touched entries. The hot loop performs no allocation, no hash lookups, and no dynamic dispatch. The cache stores exact unweighted `f32` costs so that caching does not change the arithmetic. An index-based CSR variant was tried and rejected (about 305k versus 370k deltas per second). Non-linear terms update in $O(1)$ from per-finger accumulators.

Table 1: Single-thread speed (ANSI-30, English, default profile, i9-13900H; `gecko perf`, 2026-10-21).

Operation	Rate (/s)	Time
Full evaluation	158,101	6.33 µs
Swap delta	638,805	1.57 µs
Scorer build	334	3.00 ms
Move one key, rebuild	1,551	0.64 ms

A delta is about $4\times$ faster than a full evaluation under the default profile. How much a profile costs depends on which *channels* it switches on, not on how many metrics it has (Figure 3). We enabled metrics in a fixed order and measured swap deltas after each addition. Throughput falls in three steps: from 4.8×10^6 per second with three bigram and gap-1 skipgram metrics to 2.4×10^6 when the first far-skipgram metric (S02, gaps 2–3) switches on its channel, to 2.1×10^6 when the load and balance terms add their accumulators, and to 0.64×10^6 when the first trigram metric adds the sparse trigram list. Between steps the curve is flat: metrics 4 to 13 and metrics 25 to 35 cost nothing extra. The rule of thumb for profile authors is that trigram metrics set the price of a search and the number of bigram metrics does not.

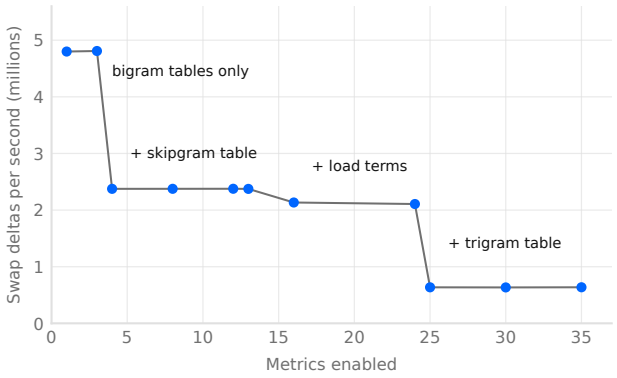


Figure 3: Swap-delta throughput against the number of enabled metrics (one thread, ANSI-30, English; median of 5 runs; script `metrics_scaling.py`). Labels name the channel each step switches on.

4.4 Adaptive top-K scoring

Because trigrams set the price, the scorer can search on a truncated trigram list and confirm on the full one. With `--topk K` the search keeps the K most frequent trigrams per symbol, adds a per-symbol bound on the dropped mass, and re-scores every candidate that enters the top list, and the final ranking, with the full objective. Reported scores are therefore always exact; only the path of the search changes. Figure 4 shows the trade: TODO(run) `gecko bench --algos sa --topk 500,2000,8000,all --seeds 30`.

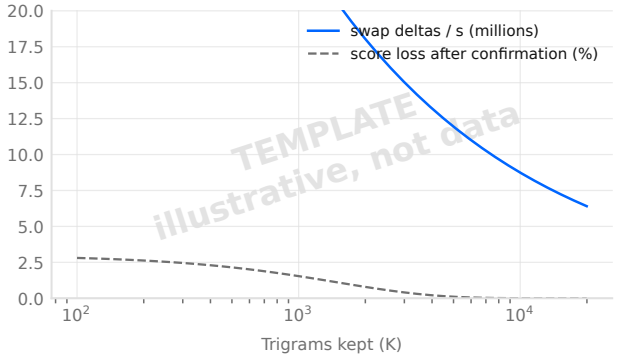


Figure 4: Template. Swap-delta throughput and final score loss (after full-objective confirmation) against the number of trigrams kept per symbol. TODO(run): `gecko bench --topk`, ANSI-30, English, default profile.

The runner scales linearly to 4 threads (Figure 5): $4.07\times$ the one-thread rate. It then bends as efficiency cores and hyperthreads join: 12 threads reach $8.3\times$ and 20 threads $8.7\times$ (5.0×10^6 evaluations per second; the CPU has 6 performance and 8 efficiency cores). With top-K scoring on 20 threads the search runs at TODO(run) evaluations per second.

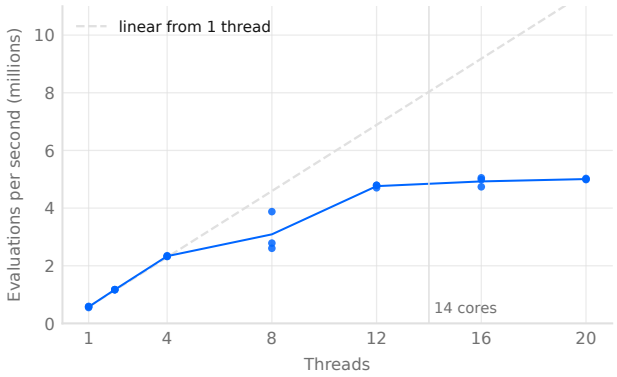


Figure 5: Evaluations per second of simulated annealing against threads (3 seeds per point, 4 s each; ANSI-30, English, default profile, full trigram list; i9-13900H with 6 performance and 8 efficiency cores; script `thread_scaling.sh`).

4.5 Invariants

Correctness rests on tests that run in CI.

- **delta = full**: for random layouts and swaps, the delta equals the re-evaluation within 10^{-5} relative, including 36-key boards with tokens and empty slots (property test).
- **folded = unfolded**: the folded score equals the sum of individually computed metrics.
- **hand-computed metrics**: each metric on 3–6 key boards with answers worked by hand.
- **ranking and published numbers**: modern layouts rank above Colemak, Dvorak, and QWERTY; SFB values are within tolerance of published ones.
- **brute force**: every optimizer, and the exact solver, finds the optimum on small instances checked by enumeration.
- **rebuild = build**: re-costing a moved board equals building it from scratch, bit for bit.
- **native = WASM**: every reference layout under every profile scores the same in the browser and the CLI (relative 10^{-6}); the TypeScript and Rust hand IK agree pose for pose.
- **closed meshes**: every bundled board's plate and case STL is a set of closed shells.
- **determinism**: the same seed gives the same result.

5. Optimizers

All discrete optimizers implement one trait:

```
trait Optimizer {
  fn init(&mut self, scorer, start, rng);
  fn step(&mut self, scorer, rng)
        -> StepInfo;      // a batch of work
  fn best(&self) -> &[Scored<Layout>];
}                           // de-duplicated top-K
```

The runner owns time budgets, threads, progress events, telemetry, and persistence; algorithms contain only search logic. Workers run on scoped standard-library threads. The scorer is shared immutably, and each worker owns its state and an RNG seeded from the run seed and the worker index. A run can be paused, resumed, or cancelled without losing results. Any key can be pinned (for example the thumb keys), and every move respects pins.

Table 2: The optimizers.

Algorithm	Role
Hill climbing	baseline; inner loop of the others
Simulated annealing	main workhorse; T_0 set from sampled swap deltas
Breakout local search	descent plus adaptive kicks; samples the local-optima network
Tabu (RoTS)	robust tabu search for QAP
Genetic, memetic	crossover plus local search on islands
MAP-Elites	quality-diversity grid over behaviours
NSGA-II, random-weight SA	Pareto fronts over chosen metrics
Branch and bound	exact optimum over k free symbols
CMA-ES (IPOP)	continuous board parameters and key poses

Beyond swaps, `--moves` adds same-finger column exchange, 3-cycles, and depth-2 swaps. All are swap sequences, so they reuse the delta machinery and respect pinned keys. Larger moves give better optima per descent but cost 3–60× more evaluations, so swap-only stays the default.

5.1 Benchmarking and telemetry

`gecko bench` runs every algorithm on the same instances, seeds, and budgets and writes a leaderboard, performance profiles, and time-to-target curves; the web UI's benchmark view draws them from the server. Every worker also logs its convergence, temperature, and acceptance rate, so a run can be replayed as a process (``when did SA stop improving?'') rather than read as one number.

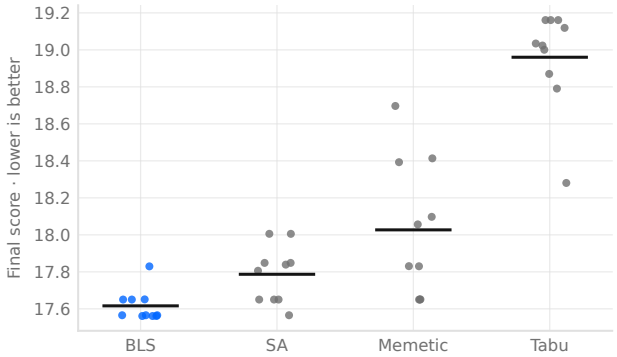


Figure 6: Final score per seed for four optimizers: 10 seeds, 2 s, 4 threads, ANSI-30, English, default profile (`bench/memetic-2026-10-21`). Black bars are means; background builds were running.

On a short budget BLS has the lowest mean (17.616 ± 0.081), followed by SA (17.787 ± 0.145), memetic (18.027 ± 0.352), and tabu (18.960 ± 0.256). Figure 7 generalizes this to many budgets and instances: TODO(run) `gecko bench --algos hc,sa,bls,tabu,memetic --seeds 30 --time 1s,5s,30s` on ANSI-30 and Corne under three profiles.

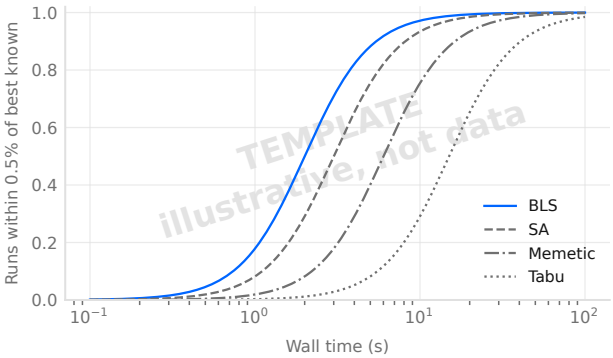


Figure 7: Template. Time-to-target curves: share of (seed, instance) runs that reach within 0.5% of the best known score by a given wall time. TODO(run): `gecko bench`, 30 seeds, 3 budgets, 2 boards, 3 profiles.

6. Explaining, bounding, and mapping

6.1 Explaining a score

For any layout, `gecko score` and `analyze` and the web UI report each metric's weighted contribution and raw value, finger and hand loads, a per-key heat map, the worst bigrams, skipgrams, and trigrams, and (with a word corpus) the worst real words per metric. Reference layouts are scored in the same pipeline and checked against published numbers (QWERTY SFB 6.63%). The web UI is the same core compiled to WebAssembly, so a number shown while dragging a key equals the CLI's; hovering a key poses the hand on it and shows the reach cost.

6.2 Lower bounds and certified sub-problems

The pair part of the score is a QAP. Each trigram is relaxed into pair terms by taking, for each pair, the minimum over the free third key, so the result remains a lower bound. The bound is Gilmore–Lawler (a linear assignment for each placement, via the Hungarian algorithm) strengthened by 20 iterations of dual ascent. Every optimization run reports the gap, $(\text{value} - \text{lower})/|\text{value}|$.

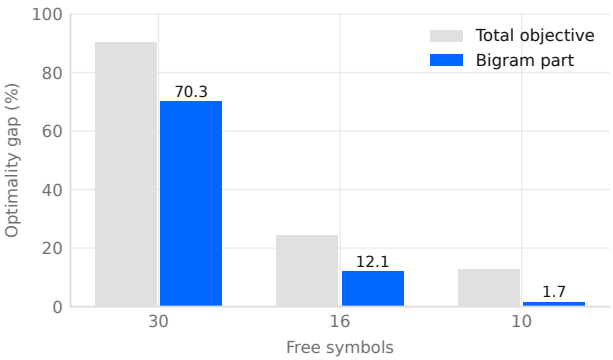


Figure 8: Optimality gap of the SA optimum against the number of free symbols (pinned symbols are constants). ANSI-30, English, default profile, run `20261021-171218-sa-a3306812`.

With all 30 symbols free, the bound certifies little

(90.3% gap). With 10 free symbols the bigram part is certified within 1.7%, and the remaining total gap (12.9%) comes mostly from the trigram relaxation. A weak bound does not mean a weak search, and `gecko certify` tells the two apart. It keeps all but k symbols of a run's best layout on their slots and finds the best placement of the k free ones exactly, by depth-first branch and bound over the dual-ascent form, with exact leaves scored by the real `Scorer`. On the run above, with the same 10 symbols free, it proved in 0.03 s (290 nodes) that SA had found the optimum: both gaps were entirely bound weakness. With the k most frequent letters free, the full objective is certified up to $k = 20$ (2.4×10^{18} placements) in 74 s, and the pair part up to $k = 22$ in 66 s (Figure 9). Every certified sub-problem so far agrees with the heuristic optimum.

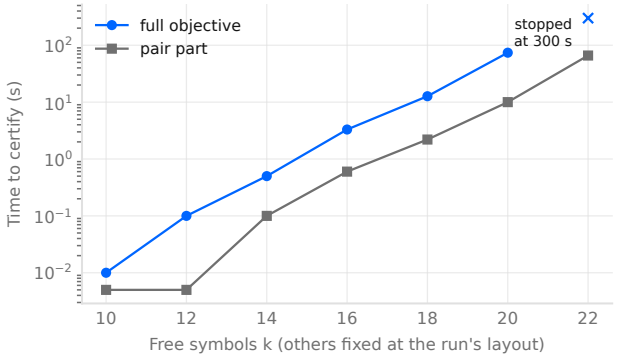


Figure 9: Time for `gecko certify` to prove the optimum of k free symbols, the rest fixed at the SA run's layout (one thread; run `20261021-171218-sa-a3306812`). Pair times below 0.01 s are drawn at 0.005 s.

6.3 Local-optima atlases and networks

`gecko explore` samples the random-score distribution and random-walk autocorrelation, then runs breakout local search with an archive that stores every local optimum with its full raw metric vector, and every kick as an edge of a local-optima network. *Exclusion restarts* begin new descents at least 8 slots from the known best, to find layouts different from the winners. Because the archive stores raw vectors, it can be *re-ranked under new weights in milliseconds* without a new search, which makes weight-sensitivity studies cheap. `gecko atlas` reports families, consensus slots, funnels, and fitness–distance correlation, and draws the network: which optima drain into which, and how symbols migrate along the path from one layout to another (Figure 10). A scored path between two layouts, for example from QWERTY to a run's best, shows the cheapest order of swaps for a user who wants to learn a layout in steps.

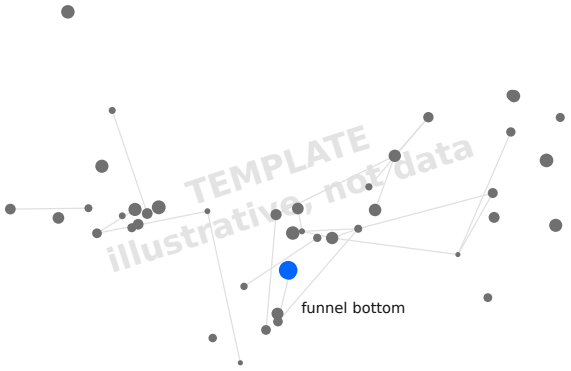


Figure 10: Template. Local-optima network of an explore run: nodes are local optima (size: basin), edges are kicks that move from one to another, colour marks the deepest funnel. TODO(run): `gecko explore --time 5m` then `gecko atlas <run> --network`.

6.4 Quality diversity, Pareto fronts, and selection

`gecko qd` runs MAP-Elites over behaviour axes (roll ratio, index load, pinky load, hand balance, or any metric id). Each cell of the grid holds the best layout with that behaviour, so a user can ask for ``more rolls'' or ``less index load'' and see the price (Section 9). NSGA-II and random-weight SA provide Pareto fronts over chosen metrics, drawn in the web UI. `gecko select` gathers every layout from any set of runs (ranked lists, fronts, atlases, QD grids), ranks them per metric, and sorts by the sum of ranks: a layout that is good on everything beats one that wins a single metric, without trusting the profile's weights.

7. Geometry in the loop

7.1 Boards

Boards are TOML files: explicit key lists, or parametric models (zones, columns, rows, stagger, splay, split, thumb arcs, row and column curvature, column lift, tent, and tilt). They import from keyboard-layout-editor.com, and eleven are bundled (ANSI-30 with standard and angle-mod fingering, ortho-30, Corne, Sweep, Lily58, Kyria, ErgoDox, Dactyl-Manuform 5×6, and parametric Corne and ortho models). Efforts and stretches are derived from key poses, so they follow moved keys.

7.2 The hand model

Geometry-dependent costs come from a *hand model*: a skeleton placed on the home keys, finger inverse kinematics with joint limits, a 4-DoF thumb, and arm geometry. It produces per-key costs for reach, press-angle alignment, wrist posture, and thumb-arc fit. These are per-slot tables, so they fold into the scorer with no search-time cost. In the web UI the hand has size presets and every segment length can be edited, so a score can be tied to one person's hand.

Hand size changes the answer. On ANSI-30 the hand

profile cuts reach cost by a third (0.71 → 0.47) against the default optimum, and its optimum costs only 1.9% under the default profile, while the default optimum costs 9% under the hand profile. A small hand gives up more reach (0.61); a large hand moves frequent letters to the middle and ring top row it reaches easily (runs `20261021-215352-sa-*`). For a board shared by several people, or a product, `--hands small,medium,large` optimizes the mean or worst-case cost over a set of hands instead of one (Figure 11).

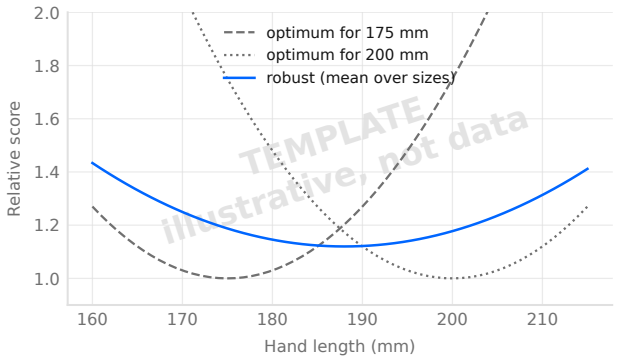


Figure 11: Template. Score across hand lengths for layouts optimized for one hand size and for the mean over sizes. TODO(run): `gecko optimize -p hand --hands 160:215:5 --robust mean` on Corne, English.

7.3 Finger assignment

Which finger presses a key is part of the board, not a law: the angle mod on row-staggered boards moves the left bottom row one column over. GECKO scores any fixed assignment (ANSI-30 is bundled with both), and `--fingers search` adds the assignment to the search: each key lists the fingers allowed to press it, and a move changes one key's finger, re-costing only that key's tables. TODO(run): `gecko optimize -g ansi-30 --fingers search` against the fixed angle-mod board, Table 3.

Table 3: Fixed and searched finger assignments (ANSI-30, English, default profile). TODO(run).

Fingering	Best score	SFB %	Stretch %
Standard	17.56	0.51	TODO
Angle mod	TODO	TODO	TODO
Searched	TODO	TODO	TODO

7.4 Board and layout search

`Scorer::set_geometry` recomputes only the tables whose inputs moved and is bit-identical to a full rebuild. Moving one key costs 0.64 ms against 3.1 ms on ANSI-30 ($4.8\times$), 0.78 ms against 4.6 ms on Corne, and 2.2 ms against 19.5 ms on the 3D Dactyl-Manuform ($9\times$).

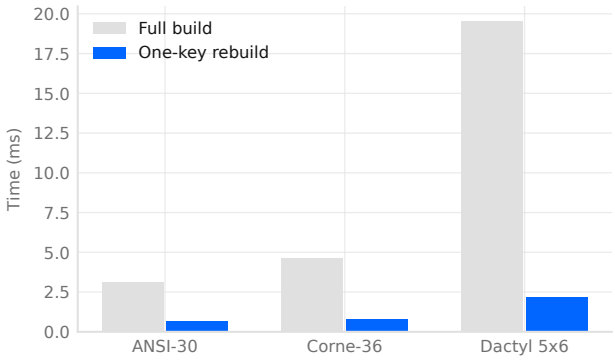


Figure 12: Time to re-cost the scorer after moving one key, against a full build (`gecko perf`, English, default profile).

This makes bilevel search feasible. `gecko geometry optimize` runs CMA-ES with IPOP restarts over chosen board parameters (a 2d preset, a 3d preset of 34 parameters, or named fields with bounds). Each candidate board gets a short warm-started layout search from the incumbent layout, all candidates of a generation share one inner seed (common random numbers), and each thread owns a scorer. An *alternate* mode instead fixes the layout, moves the board, and repeats; a *board* mode shapes a board for a layout the user already types. Default bounds encode posture priors from the ergonomics literature (a slight negative tilt, a split opening near 15°), and the user can widen them.

On Corne with the hand profile, a 60 s 2D search improves the score by 9–11% over the start board at equal layout effort, and a 120 s 3D search by 36–37% (Figure 13). The 3D gain is the hand model's: wrist posture falls from 1.67 to 0.02, press angle from 1.84 to 0.40, and reach from 1.26 to 0.59 (run `20261022-174027-geom-10c6ce3e`). Every 3D run tents the halves 26–40° and tilts them away from the user, the direction the posture literature recommends. Row and column curvature trade off, so the optimum is a plateau of shapes, not one shape.

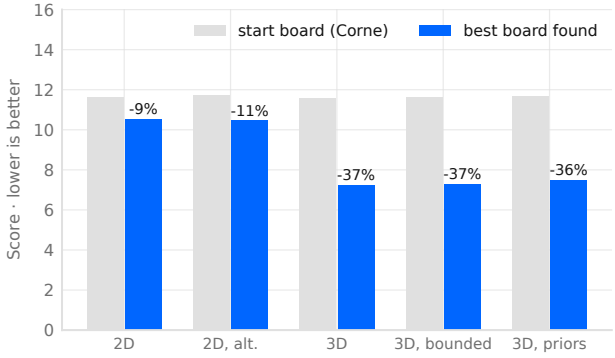


Figure 13: Board + layout search on Corne (hand profile, English, QWERTY start, 20 threads): start board and best board, each after the same final layout search. 2D: 60 s; 3D: 120 s; ``bounded'' and ``priors'' restrict curvature, tent, and tilt (`context/algorithms/geometry-search.md`).

7.5 Free-form boards and thumb clusters

Parametric boards share one shape per column; `--free keys` lets every key move in all six degrees of freedom, starting from the best parametric board. A smoothness term keeps neighbouring keys in a printable surface and a repair step pushes apart any that collide. Two refiners are offered: CMA-ES, and a gradient method on a differentiable surrogate of the reach and posture tables, which is faster when the parametric start is already good (Figure 14). The thumb cluster is searched the same way, including how many thumb keys it has: each added key must pay for itself in score.

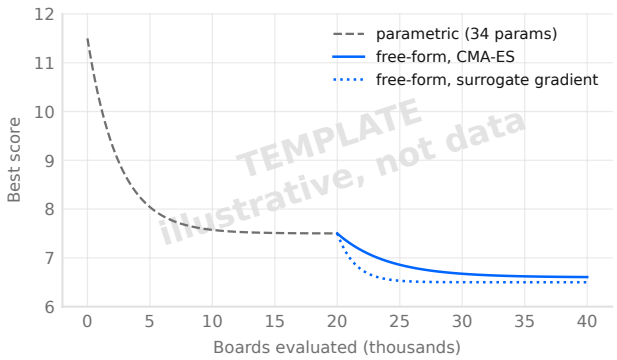


Figure 14: Template. Best score against boards evaluated: parametric 3D search, then free-form refinement with CMA-ES or with the surrogate gradient. TODO(run): `gecko geom optimize --free 3d`, then `--free keys --refine cmaes|gradient`, Corne, hand profile.

8. From model to print

A board is only useful if it can be built. Every candidate is checked for *collisions* in 3D: keycap, switch body, and the volume a finger needs to reach the key, tested pairwise with GJK and EPA over a spatial grid. Overlap enters the objective as a penalty proportional to depth, and boards that overlap past a hard limit skip their layout search. A profile's `[build]` section sets *manufacturability* limits (key angle, height above the desk, the size of each printed piece against a print bed), and they share the same penalty. The search reports the best collision-free board.

`gecko build stl run:<id>` turns key poses into a printable board: a switch plate with MX or Choc cutouts and clip pockets, webs and junctions between neighbouring keys, walls down to the desk, a bottom plate, screw bosses, stabilizer cutouts for long keys, and openings for the controller, USB, and the cable between halves. Each part is a list of closed convex shells, so any slicer can print it directly, and the parts are checked against switch travel and keycap clearance. The web UI's Export tab renders the same case, from the same core, and downloads the files. Corne, Kyria, Dactyl-Manuform, and a 3D-optimized Corne all union into single manifold solids.

9. Using GECKO

Each use below is a question, the commands that answer it, and what they returned on the bundled data (ANSI-30 or Corne, English, default profile unless stated). Runs and scripts are in `bench/software-paper-2026-10-21/`.

Table 4: Uses at a glance.

Question	Commands	You get
Why does a layout rank where it does?	`compare`, `score`, `analyze`	per-metric and per-word costs
Best layout for my board and language?	`optimize`, `certify`	ranked layouts, bound, holdout check, certificate
How good is ``good''?	`landscape`, `explore`, `atlas`	random baseline, optima, families, network
What does a goal cost?	`qd`, `optimize --algo nsga2`, `select`	best layout per behaviour cell
Can a layer or adaptive key help?	`score`, `magic`	score with rules applied
Which board suits my hand?	`geom optimize`	board, layout, and their costs
Can I print it?	`build stl`	plate, case, build report
Is this result reproducible?	`verify`, `layouts`	pass or fail; exchange files

9.1 Audit a layout and see why it ranks

`gecko compare` ranks the ten bundled reference layouts under one profile and one corpus (Figure 15). Under the default profile the order is Graphite, Sturdy, Semimak, Canary, Dvorak, Colemak-DH, Halmak, Colemak, Workman, QWERTY. This ranking is a property of the profile, not a claim about comfort, and the per-metric table says what drives each gap: Colemak and Colemak-DH have identical SFB (1.54%), rolls-in, and redirect rates, and differ in lateral stretch (4.52% against 2.05%), which accounts for the visible part of their 2.1-point gap. `gecko analyze --words` goes further and ranks real words by their share of a metric's cost. For Graphite on the `shai` word corpus, the lateral-stretch cost is dominated by one word: ``you'' carries 21% of it and ``your'' another 12%. A designer who sees this can decide whether the metric or the layout is at fault, which a single score cannot say.

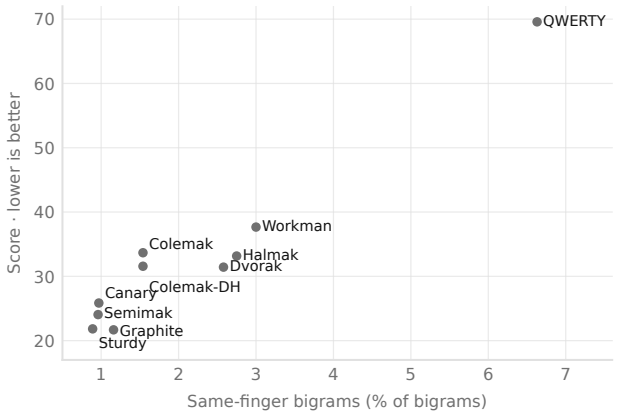


Figure 15: Reference layouts under the default profile (`gecko compare`, ANSI-30, English): score against same-finger bigram rate. Lower is better on both axes.

9.2 Design a layout for a board and a language

`gecko optimize` writes a run directory and prints the ranked layouts, the per-metric breakdown of the best, and a lower bound. On ANSI-30 with the default profile a 10 s search on 20 threads (52.6M evaluations, seed 7) found a layout scoring 17.561 against Graphite's 21.680 under the same profile. Figure 16 shows what it bought and what it paid: SFB falls from 1.16% to 0.51% (the largest single gain), stretch, SFS, and movement improve, and the layout pays in redirects (4.7% against 2.7%) and effort. Seeing the price is the point of the breakdown: a different profile would pay differently. `gecko certify` then checks the result: for an SA optimum of the same score (run `20261021-171218-sa-a3306812`), no placement of the 20 most frequent letters, the other ten fixed, does better.

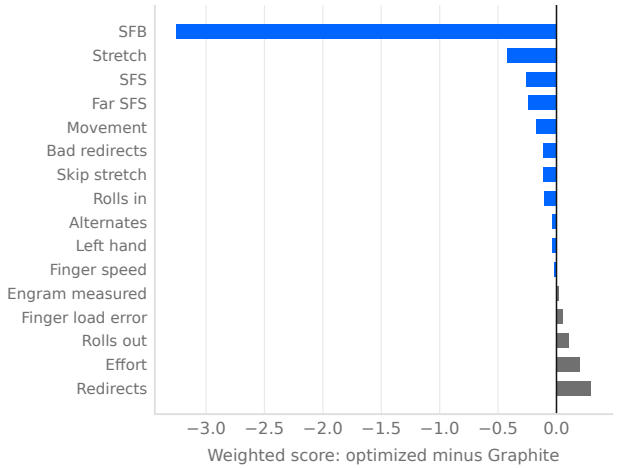


Figure 16: Per-metric weighted score of the optimized layout minus that of Graphite (same profile; metrics with non-zero weight). Left of zero is better. Run `20261022-031427-sa-8b3885fd`.

The same command designs for other boards. On the 36-key Corne with the five thumb keys pinned (`--pin '{space}{enter}…'`), 10 s of simulated annealing ran at 4.7M evaluations per second and had reached its final layout 2.7 s in. Scores on different boards are not comparable, because the geometry changes the cost tables. For programming, a layered Corne layout (symbol and number layers) is scored with the `code` profile and corpus, which add the layer-switch, symbol-bigram, and paired-bracket metrics.

A design is only as good as the corpus it was fitted to. `optimize --holdout shai` re-scores every ranked layout on a second corpus the search never sees. For the run above the best layout keeps 99.6% of its gain on the held-out corpus (start 70.2 $\rightarrow$ 18.3). The check also shows a limit: the top ten layouts span only 0.27 points on the search corpus (17.56 to 17.83), and their order has rank correlation 0.16 on the held-out one, so differences among near-ties should be settled by the breakdown, or by `select`, not by rank.

9.3 How good is ``good''?

`gecko landscape` samples 2000 random layouts: scores are 98.4 ± 17.5, the random-walk correlation length is 8.5 swaps, and the best random layout scores 49.6. Of the six reference layouts it places among the samples, five lie more than 3.4 standard deviations below the random mean and no random sample beats them; QWERTY is 1.65 below, and 3.1% of random layouts beat it. The optimum we found (17.56) is 4.6 standard deviations below. `explore` then maps the good region: it archives local optima, groups them into families, reports consensus slots, and draws the network between them. Because the archive keeps raw metric vectors, `atlas -p <profile>` re-ranks every optimum under another profile in milliseconds, so ``would a different weighting change the winner?'' is a query, not a new search.

9.4 See the price of a preference

Suppose a user wants more rolls. `gecko qd --axes roll_ratio,index_load` fills a grid with the best layout for each combination (Figure 17). In 20 s and 107M evaluations it filled 92 of 144 cells (64%). The best layout sits near 55% rolls and 27% index load. The best layout found with 65% rolls costs only 0.4 points more (17.93 against 17.56), but with 69% the cost is 2.6 (15%) and with 72% it is 5.5. The user chooses a point on the curve instead of accepting one optimum. `optimize --algo nsga2 --objectives B01,T03,M01` gives the same information as a Pareto front over named metrics, and `select` picks across all of these runs at once.

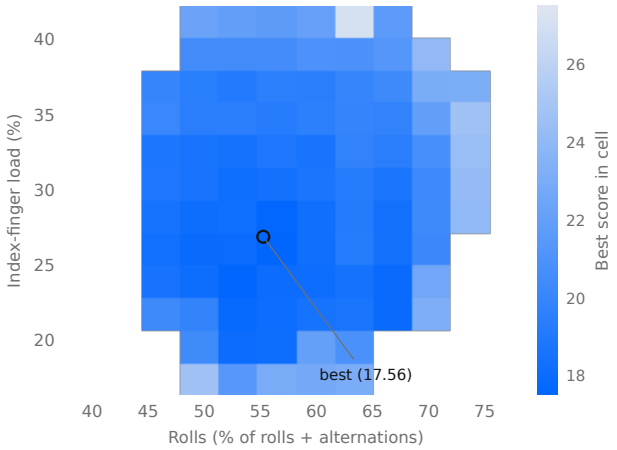


Figure 17: MAP-Elites map (ANSI-30, English, default profile, 20 s, 12×12 cells): best score per cell; blank cells were not reached. Run `20261022-031516-qd-af3fe56a`.

9.5 Try a layer or an adaptive key

Layers and adaptive (``magic'') keys are part of the layout model, so they are scored by the same tables. A magic key outputs a letter that depends on the previous one; `gecko magic` searches the rules greedily on a fixed layout, adding the rule that lowers the score most and dropping rules that stop paying. The bundled Magic Sturdy (one magic key, 14 rules) scores 19.995 against 21.831 for Sturdy, and SFB falls from 0.89% to 0.27%. The model charges one press for a magic output, so this is a prediction of comfort, not a measurement. `optimize --from` a magic layout then moves the key like any symbol with the rules fixed, and the two commands alternate to co-optimize layout and rules.

9.6 Design a board for one hand, and print it

A user measures their hand (palm, finger, and thumb segment lengths) in the Hand tab or a TOML file, chooses a parametric start such as Corne, and runs one pipeline:

```
gecko geom optimize -g corne-36.param \
  --free 3d --hand me.toml -p hand --time 5m
gecko geom optimize -g run:<id> --free keys
gecko certify run:<id> --free 16
gecko build stl run:<id> --switch choc
```

The first search shapes a 3D board and its layout together (Figure 13); the second refines key poses free-form; `certify` checks the layout on the final board; `build stl` writes the plate and case. Every step is a run directory whose inputs point at the previous run, so the chain from a hand to a printed board is one reproducible record that `gecko verify` can re-execute end to end. TODO(run): the full walkthrough with a measured hand, its timings, and a photo of the printed board.

9.7 Reproduce, exchange, and publish

Any run made with an evaluation budget, layout or board, can be re-run from its directory: `gecko verify run:<id>` checks the input hashes, re-executes with the recorded parameters, and compares evaluations, every ranked layout, and the best board. On a 3M-evaluation SA run it printed `reproduced`. Layouts move between tools: `layouts import` reads Oxeylyzer, genkey, and cmini files (reporting dropped keys as warnings), `layouts export` writes them back, `geometry import-kle` reads keyboard-layout-editor boards, and `layouts svg` draws the image the web Export tab produces, with finger colours or a frequency heat map. A GECKO score can therefore be checked against another tool's on the same layout, and a result can be shared as one file.

10. Related work

Table 5 summarizes the tools we read (details and sources in `research/studies/tool-comparison.md`). They differ in what they judge and how they search. None of the layout analyzers we read scores a layout on an arbitrary 3D board with a hand model, and none optimizes board and layout together; the board generators shape keys without judging typing. Integer programming (Karrenbauer and Oulasvirta) certifies optimality on the full problem for its objective; GECKO's full-board bound is weaker, and it certifies sub-problems exactly instead (Section 6). For search we use ideas from QAP heuristics (robust tabu search [1], breakout local search [2]), Gilmore–Lawler bounds [3], multi-objective search [4], quality diversity [5], and CMA-ES [6]. For costs, we draw on finger independence [7], large-scale typing timing [8], and the Engram comfort study.

Table 5: Tools compared (from documentation and source). ``Geometry'' is what the tool can score on.

Tool	Judges	Search	Geometry
carpalx	triad effort model	annealing	fixed 2D
Oxeylyzer	SFB/SFS, stretch, trigram classes	greedy, depth-N, annealing	fixed 2D
genkey	finger speed, LSB	greedy, multi-swap	fixed 2D
KLO	movement table, trigram irregularity	annealing, GA	fixed 2D
Kühlmak	bigram classes, balance	threshold annealing	fixed 2D
Engram	measured pair comfort	exhaustive, B&B	fixed 2D
Integer programming	ergonomics, familiarity	IP with bounds	fixed 2D
Cosmos, ergogen	none (generate boards)	manual, parametric	any
GECKO	48 metrics, hand model	9 optimizers, QD, B&B, CMA-ES	2D, 3D, free-form; STL

11. Reproducibility and engineering

Seeded RNG. xoshiro256++ is implemented in-tree and is identical on native and WebAssembly builds; CMA-ES uses it too, so board searches reproduce in the browser. **Run directories** (`runs/<time>-<algo>-<hash>/`) hold `config.json`

(parameters, input names and content hashes, the full profile, start layout or board, pins, version), `progress.jsonl` and per-worker telemetry, and `best.json` (top layouts or boards with full breakdown, evaluations, speed, optimality gap); board runs add the best board as `geometry.toml`, and builds add `stl/`. `gecko verify run:<id>` re-runs an evaluation-budget run and checks hashes, evaluation count, and results; time-budget runs are refused because they are not deterministic. **Content hashing** (FNV-1a of geometry, corpus, profile tables, and layouts) de-duplicates results and detects stale atlases. **Stable JSON**: every CLI command supports `--json` with a versioned schema, and a contract test compares the web client's types against real server output. **CI** runs the Rust tests, clippy with warnings as errors, and rustdoc, plus the web type check, unit tests, the WASM parity test, and the contract test. Benchmarks append to `bench/history.jsonl` with the commit hash; curated experiments, including those in this paper, live in `bench/` with the scripts that produced them. **Size**: about 28k lines of Rust (24k in the core) and 9.3k of TypeScript; about 250 Rust tests and 22 web test files. TODO(run): coverage.

12. Limitations

Costs are models, not measurements. Scores predict comfort; they are not observed comfort or speed, and no user study has yet tied a GECKO optimum to faster or more comfortable typing. The hand model's constants (curl, joint coupling, wrist-posture optimum) come from the literature and are not fitted to a population, so 3D boards are optimal for the model, and the default bounds on tent, tilt, and curvature matter as much as the search. The full-board lower bound is weak; sub-problems of up to about 20 free letters are certified exactly, the full 30 are not. A learned-profile experiment gained only 1.8 points of pairwise accuracy (53.8% against 52.0%), and community likes are confounded by popularity. Held-out corpus checks keep the gain but do not preserve the order of near-tied layouts. Corpora are dominated by English; multilingual and code corpora exist but are less validated. The adaptive-key model has no typing data behind it. All speed numbers come from one hybrid-core laptop CPU.

13. Availability

GECKO is MIT-licensed. Build with `cargo build --release`; run the web UI with `gecko serve`; the WASM package is built by `npm run wasm`. Corpora, boards, hands, profiles, and reference layouts ship in `data/`. A first session:

```
gecko compare
gecko analyze graphite -c shai --words
gecko optimize --time 30s
gecko certify run:<id> --free 12
gecko qd --axes roll_ratio,index_load
gecko geom optimize -g corne-36.param \
  --free 3d -p hand --time 2m
gecko build stl run:<id>
gecko verify run:<id>
```

References

[1] É. Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17, 1991.

[2] U. Benlic and J.-K. Hao. Breakout local search for the quadratic assignment problem. *Applied Mathematics and Computation*, 2013.

[3] P. C. Gilmore. Optimal and suboptimal algorithms for the quadratic assignment problem. *J. SIAM*, 10, 1962. E. L. Lawler. The quadratic assignment problem. *Management Science*, 9, 1963.

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolutionary Computation*, 6(2), 2002.

[5] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.

[6] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 2001.

[7] C. Häger-Ross and M. H. Schieber. Quantifying the independence of human finger movements. *J. Neuroscience*, 20(22), 2000.

[8] V. Dhakal, A. M. Feit, P. O. Kristensson, and A. Oulasvirta. Observations on typing from 136 million keystrokes. *Proc. CHI*, 2018.